



SWEN 221

A surreal, dreamlike landscape. In the center, a person stands on a small, dark, rocky outcrop that appears to be part of a vast, ethereal sea of clouds. The person is looking out over a vast, colorful sky. The sky is filled with large, billowing clouds in shades of pink, purple, blue, and yellow. A bright, glowing light source, possibly a sun or moon, is visible on the horizon, casting a warm glow over the scene. The sky is also filled with stars, a comet streaking across the upper left, and a large, glowing nebula or galaxy in the upper right. The overall atmosphere is one of wonder and exploration.

Marco Servetto
CO258

www.youtube.com/MarcoServetto



A Minecraft (bug) story

To 'if' or not to 'if'

```
//Marco said to avoid ifs...
int myCode1(boolean condition){
    //..
    if (condition){ return code1(); }
    return code2();
}
```

```
//no problem, I can just do this...
int myCode2(boolean condition){
    //..
    return Map.<Boolean,Supplier<Integer>>of(
        true,()->code1(),
        false,()->code2())
        .get(condition).get();
}
```



To 'if' or not to 'if'

```
int myCode1(boolean condition){  
    //..  
    if (condition){ return code1(); }  
    return code2();  
}
```

```
//no problem, I can just do this...  
int myCode2(boolean condition){  
    //..  
    return Map.<Boolean,Supplier<Integer>>of(  
        true,()->code1(),  
        false,()->code2())  
        .get(condition).get();  
}
```

//Actually... both version are equally bad in the sense that
//both versions are hiding the decision in the method body
//and there is no way for the user to customize it.

//Now I will tell you a story, to explain why they are both bad,
//and in turn, why the 'if' is bad.

A story of minecraft Skeletons

- Originally in Minecraft that was only one kind of skeleton:
- It was simply called “skeleton”.
- When a skeleton is killed by a charged creeper, it would drop a skeleton head.



A story of minecraft Skeletons

- Then, minecraft added “wither skeletons”
- When a wither skeleton is killed by a charged creeper, it would drop a wither skeleton head.



A story of minecraft Skeletons

Wither Skeleton heads are very important in the game, they can be used to summon the Wither.



A story of minecraft Skeletons

- The code looked like that (pseudocode):

```
Skeleton.die:
```

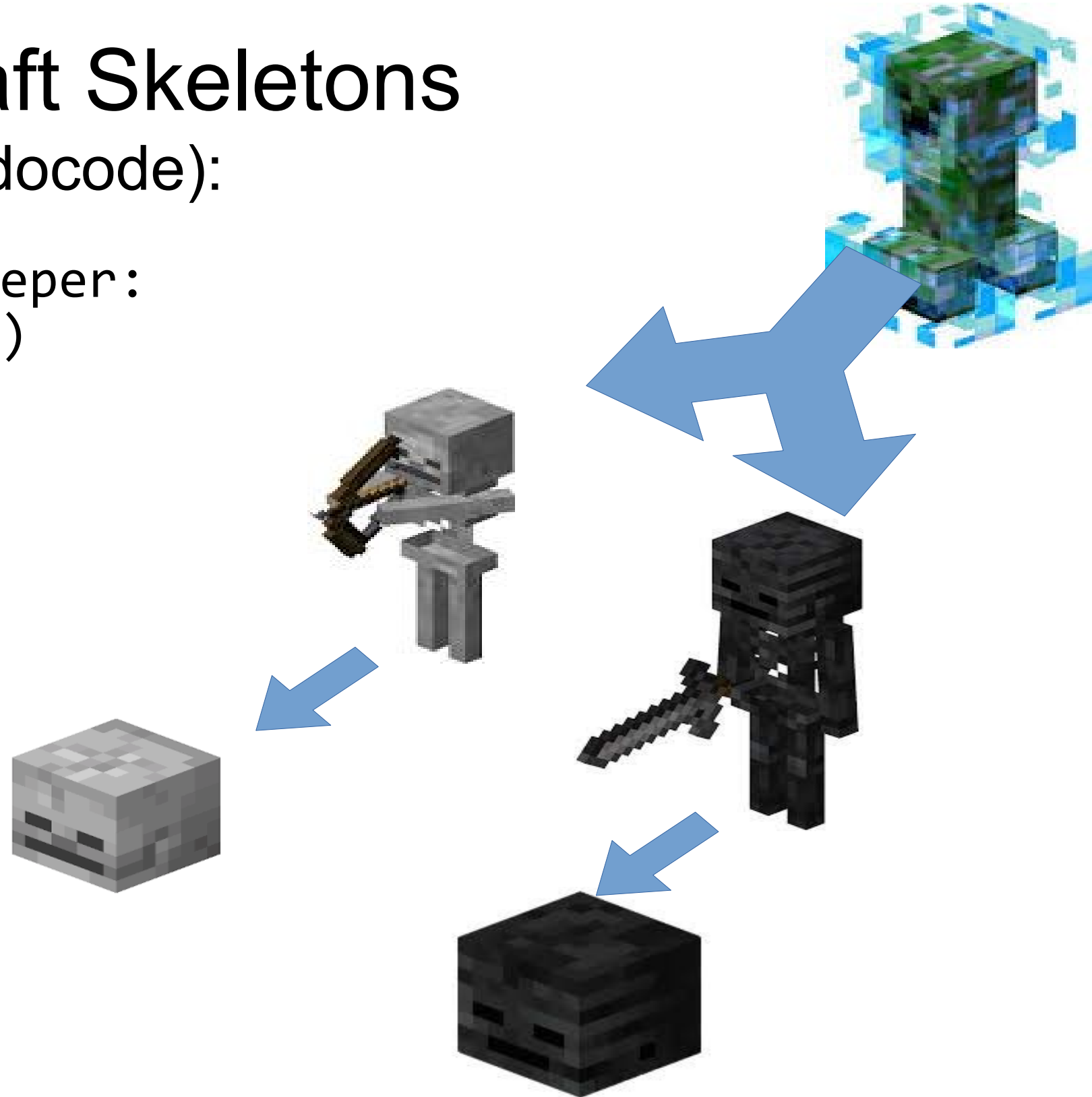
```
  when killed by a charged creeper:
```

```
    if (the_skeleton_is_normal)
```

```
      drop(skeleton_skull)
```

```
    else
```

```
      drop(wither_skull)
```



A story of minecraft Skeletons

- Many years passed, everyone forgot that specific code.
- Then they introduced a new kind of skeleton.
- The “Stray”



Hi Guys, it will be a
pleasure to work
with you!

A story of minecraft Skeletons

- Many years passed, everyone forgot that specific code.
- Then they introduced a new kind of skeleton.
- The “Stray”



Poor guy, he have no idea
what is coming...

A story of minecraft Skeletons

- Many years passed, everyone forgot that specific code.
- Then they introduced a new kind of skeleton.
- The “Stray”



Poor guy, he have no idea
what is coming...



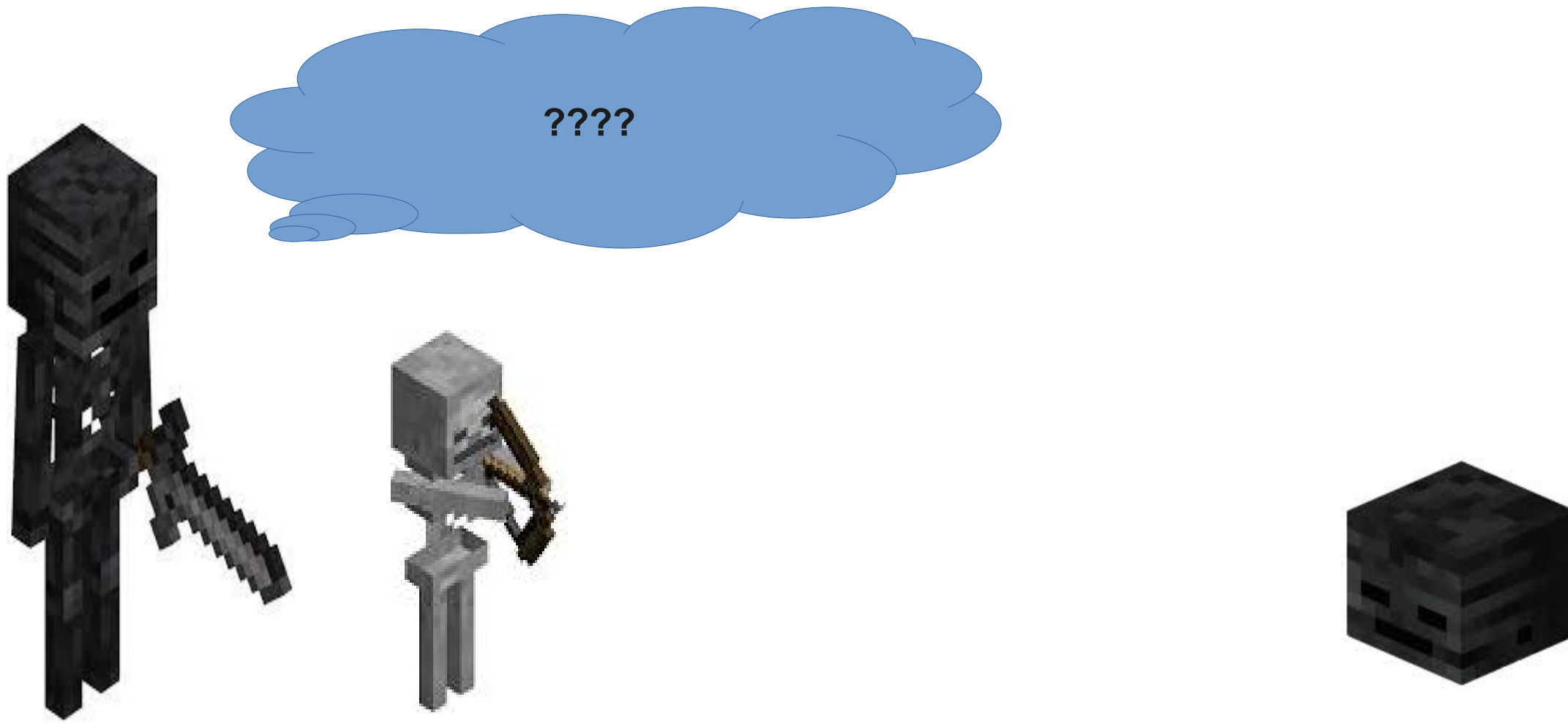
A story of minecraft Skeletons

- Many years passed, everyone forgot that specific code.
- Then they introduced a new kind of skeleton.
- The “Stray”



A story of minecraft Skeletons

- Many years passed, everyone forgot that specific code.
- <https://bugs.mojang.com/browse/MCPE-35876>



A story of minecraft Skeletons

- Compare those two different uses of 'if':

`Skeleton.die:`

```
when killed by a charged creeper:  
  if (the_skeleton_is_normal)  
    drop(skeleton_skull)  
  else  
    drop(wither_skull)
```

`Skeleton.die:`

```
when killed by a charged creeper:  
  if (the_skeleton_is_wither)  
    drop(wither_skull)  
  else  
    drop(skeleton_skull)
```

A story of minecraft Skeletons

- Compare those two different uses of 'if':

```
Skeleton.die:  
  when killed by a charged creeper:  
    if (the_skeleton_is_normal)  
      drop(skeleton_skull)  
    else  
      drop(wither_skull)
```

```
Skeleton.die:  
  when killed by a charged creeper:  
    if (the_skeleton_is_wither)  
      drop(wither_skull)  
    else  
      drop(skeleton_skull)
```

If there are only two kinds of skeletons, they behave the same.

A story of minecraft Skeletons

- Compare those two different uses of 'if':

```
Skeleton.die:  
  when killed by a charged creeper:  
    if (the_skeleton_is_normal)  
      drop(skeleton_skull)  
    else  
      drop(wither_skull)
```

If there are only two kinds of skeletons, they behave the same.

```
Skeleton.die:  
  when killed by a charged creeper:  
    if (the_skeleton_is_wither)  
      drop(wither_skull)  
    else  
      drop(skeleton_skull)
```

If there are three kinds of skeletons, they behave differently!

A story of minecraft Skeletons

- Compare those two different uses of 'if':

```
Skeleton.die:  
  when killed by a charged creeper:  
    if (the_skeleton_is_normal)  
      drop(skeleton_skull)  
    else  
      drop(wither_skull)
```

```
Skeleton.die:  
  when killed by a charged creeper:  
    if (the_skeleton_is_wither)  
      drop(wither_skull)  
    else  
      drop(skeleton_skull)
```

Not a 'truly' boolean
condition: hiding a
'case analysis'

If skeletons had a 'dropSkull'
dynamically dispatched
method, this would have not
happened.

The good code

Mob.die:

```
    when killed by a charged creeper:  
        this.dropSkull()
```

abstract Mob.dropSkull() //to be overridden

```
-----  
NormalSkeleton.dropSkull():  
    drop(skeleton_skull) //overridden
```

```
-----  
WitherSkeleton.dropSkull():  
    drop(wither_skull) //overridden
```

```
-----  
StraySkeleton.dropSkull():  
    drop(skeleton_skull) //overridden
```

Easy to see in retrospective.
But in retrospective is
pointless, the bug happened
already.

- How to code so that the bug would not happen in the first place?
- Having abstract 'dropSkull' forces to add a specific behavior for each new mob!

Why ifs are bad?

- An if/switch hides a decision about classifications inside the hidden body of a method.
- When the code evolves, this decision stays hidden and slowly rot.

https://en.wikipedia.org/wiki/Software_rot

Why ifs are bad?

- On the other side, dynamic dispatch works on public* methods.
- We need a method to override in order to make our dynamic dispatch; thus this method becomes the explicit documentation point for those classification decisions!

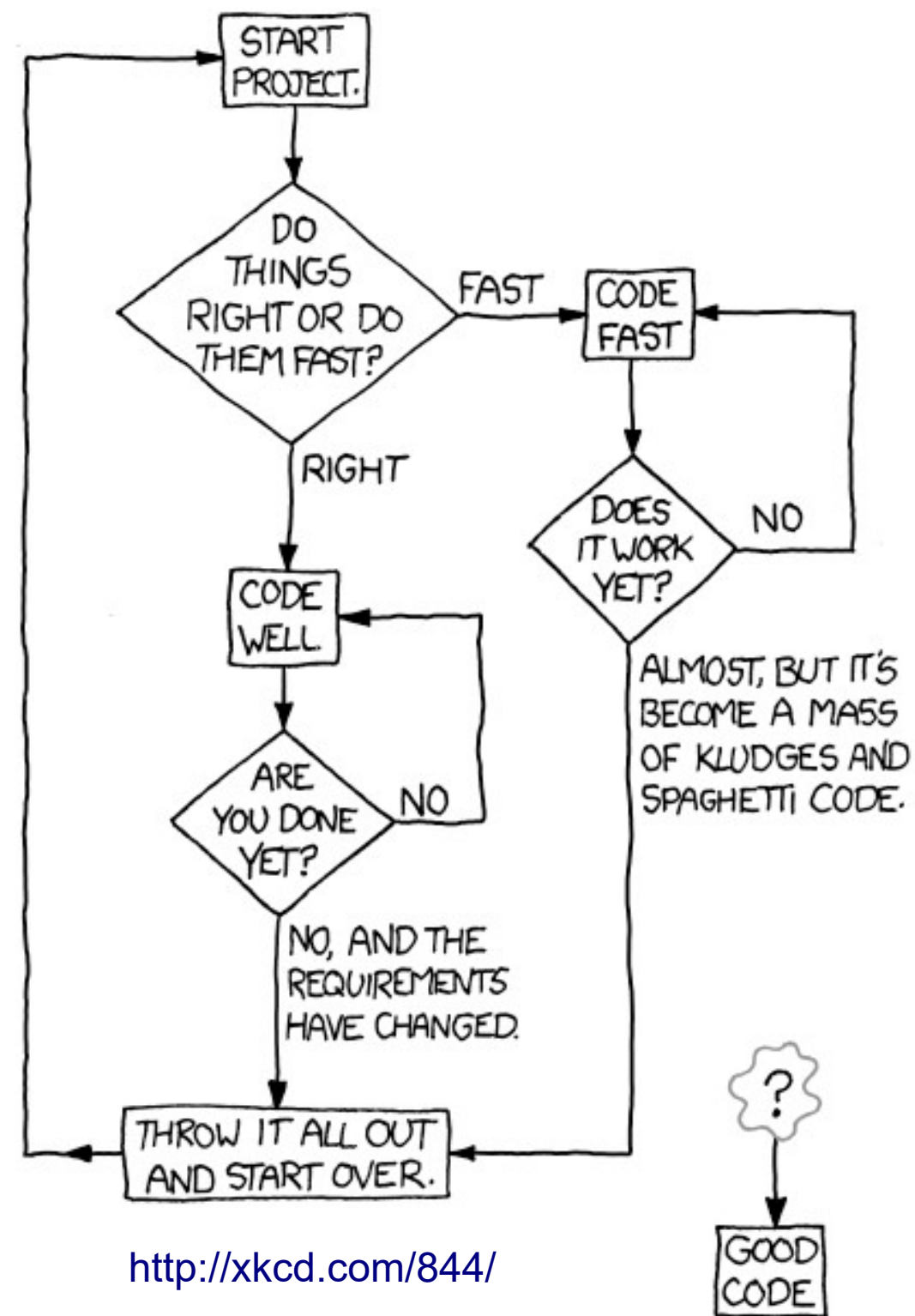


Unconditional programming





HOW TO WRITE GOOD CODE:



<http://xkcd.com/844/>

When to if?

- When a choice is really, really binary:
- A point is either inside or outside of a bounding box
- An element is inside or outside a list
- A list is empty or not
- When working with legacy code / methods returning booleans

When to if?

- When a choice is really, really binary:
 - A point is either inside or outside of a bounding box
 - An element is inside or outside a list
 - A list is empty or not
 - When working with legacy code / methods returning booleans
-
- What if we just assumed that the choice was binary, but the future proves us wrong?

When to if? (disclaimer: precognition required)

- When a choice is really, really binary:
 - A point is either inside or outside of a bounding box
 - An element is inside or outside a list
 - A list is empty or not
 - When working with legacy code / methods returning booleans
-
- What if we just assumed that the choice was binary, but the future proves us wrong?

When not to if? (disclaimer: precognition required)

- When a choice appears to be binary, or at least to range on a finite set of options, but the future then proves you wrong
- When you would like to 'if' just because you have no idea how else to code!
- When choosing an action over an open set options
- When the set of options may change over time
- When the behavior in response to an option may change over time, or may need to be customized by the user code

When to if? (disclaimer: precognition required)

- It is a way to ‘document’ at the ‘public’ level that that choice is really binary.
- It is exactly what a boolean `.isEmpty()` method is doing.
- All of ‘if’, ‘switch’, ‘try-catch’, ‘`myOpt.ifPresent(()->..)`’, and even `Map.of(true,()->a,false,()->b).get(condition).get();`
allows to bury/hard-code decisions inside of the code.
- It is not always bad, sometimes it make sense to hard-code a decision. Just... only sometimes...
- But what times?
Precognition is required to answer this question.

Taking a step back

- The problem is not the “if”, the problem is that we reached a point in our code where we are trying to explicitly hardcode a decision.
- For example, key bindings:
 - `switch(key){ case:'a'→ goLeft() ...}`
- BAD solution.
- Good solution: use a `Map<Key, Action>`

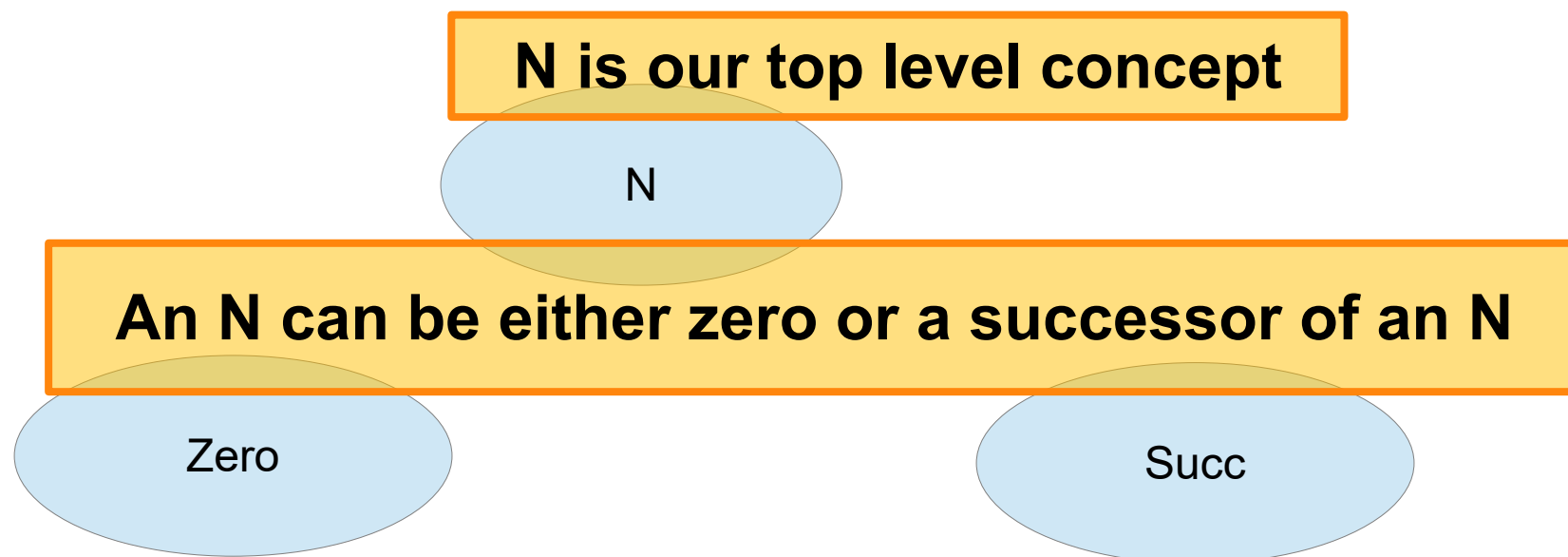
Good way to handle key-mapping

```
interface Game{  
    void goUp();  
    void goLeft();  
    void goRight();  
    void goDown();  
    void jump();  
}
```

```
class KeyMap{  
    private Game game;  
    private Map<String, Runnable> keyMap;  
    public KeyMap(Supplier<Game> newGame){  
        this.game= newGame.get();  
        Map<String, Runnable> map= Map.of(  
            //could be a separate method  
            "W", game::goUp,  
            "A", game::goLeft,  
            "S", game::goDown,  
            "D", game::goRight,  
            " ", game::jump  
        );  
        keyMap= new HashMap<>(map);  
    }  
    public void updateKey(String key, Runnable action) {  
        keyMap.put(key, action);  
    }  
    public void executeAction(String keyPress) {  
        keyMap.getOrDefault(keyPress.toUpperCase(), ()->{}).run();  
    }  
}
```

Can we really avoiding using ifs?

- Exercise: Peano numbers.
- Zero is a Number
- For every number, there is a successor number.
- We can define addition and multiplication
- We can define a toInt/fromInt so we can test it/print it.



```
public interface N {
    static N zero() { return new N({}); }
    default N add(N other){ return other; }
    default N times(N other){ return this; }
    default N succ(){
        N pred= this;
        return new N(){
            public N add(N other){ return pred.add(other).succ(); }
            public N times(N other){ return pred.times(other).add(other); }
            public int toInt(){ return pred.toInt()+1; }
        }; }
    default int toInt(){ return 0; }
    static N fromInt(int i){
        N res= zero();
        while(i --> 0){ res= res.succ(); }
        return res;
    }
}
```

```

public interface N {
    static N zero() { return new N({}); }
    default N add(N other){ return other; }
    default N times(N other){ return this; }
    default N succ(){
        N pred= this;
        return new N(){
            public N add(N other){ return pred.add(other).succ(); }
            public N times(N other){ return pred.times(other).add(other); }
            public int toInt(){ return pred.toInt()+1; }
        }; }
    default int toInt(){ return 0; }
    static N fromInt(int i){
        N res= zero();
        while(i --> 0){ res= res.succ(); }
        return res;
    }
}

```

Just to print/test

We can look at it
later




```
public interface N {  
    static N zero() { return new N({}); }  
  
    default N add(N other){ return other; }  
  
    default N times(N other){ return this; }  
  
    default N succ(){  
        N pred= this;  
        return new N(){  
            public N add(N other){ return pred.add(other).succ(); }  
  
            public N times(N other){ return pred.times(other).add(other); }  
        };  
    }  
}
```

```
public interface N {  
    static N zero() { return new N({}); } //The zero constant  
  
    default N add(N other){ return other; }  
  
    default N times(N other){ return this; }  
  
    default N succ(){  
        N pred= this;  
        return new N(){  
            public N add(N other){ return pred.add(other).succ(); }  
  
            public N times(N other){ return pred.times(other).add(other); }  
        };  
    }  
}
```

```
public interface N {  
    static N zero() { return new N({}); } //The zero constant  
  
    default N add(N other){ return other; } //0 + b = b  
  
    default N times(N other){ return this; }  
  
    default N succ(){  
        N pred= this;  
        return new N(){  
            public N add(N other){ return pred.add(other).succ(); }  
  
            public N times(N other){ return pred.times(other).add(other); }  
        };  
    }  
}
```

```
public interface N {  
    static N zero() { return new N({}); } //The zero constant  
  
    default N add(N other){ return other; } //0 + b = b  
  
    default N times(N other){ return this; } //0 * b = 0  
  
    default N succ(){  
        N pred= this;  
        return new N(){  
            public N add(N other){ return pred.add(other).succ(); }  
  
            public N times(N other){ return pred.times(other).add(other); }  
  
        };  
    }  
}
```

```
public interface N {
    static N zero() { return new N(); } //The zero constant

    default N add(N other){ return other; } //0 + b = b

    default N times(N other){ return this; } //0 * b = 0

    default N succ(){
        N pred= this; //by creating my successor, I'm the predecessor
        return new N(){
            public N add(N other){ return pred.add(other).succ(); }

            public N times(N other){ return pred.times(other).add(other); }

        };
    }
}
```

```

public interface N {
    static N zero() { return new N({}); } //The zero constant

    default N add(N other){ return other; } //0 + b = b

    default N times(N other){ return this; } //0 * b = 0

    default N succ(){
        N pred= this; //by creating my successor, I'm the predecessor
        return new N(){
            public N add(N other){ return pred.add(other).succ(); }
            //(a+1) + b = (a+b) + 1
            public N times(N other){ return pred.times(other).add(other); }

        };
    }
}

```

```

public interface N {
    static N zero() { return new N({}); } //The zero constant

    default N add(N other){ return other; } //0 + b = b

    default N times(N other){ return this; } //0 * b = 0

    default N succ(){
        N pred= this; //by creating my successor, I'm the predecessor
        return new N(){
            public N add(N other){ return pred.add(other).succ(); }
            //(a+1) + b = (a+b) + 1
            public N times(N other){ return pred.times(other).add(other); }
            //(a+1)*b = (a*b) + b
        };
    }
}

```

```

public interface N {
    static N zero() { return new N({}); }
    default N add(N other){ return other; }
    default N times(N other){ return this; }
    default N succ(){
        N pred= this;
        return new N(){
            public N add(N other){ return pred.add(other).succ(); }
            public N times(N other){ return pred.times(other).add(other); }
            public int toInt(){ return pred.toInt()+1; }
        }; }
    default int toInt(){ return 0; }
    static N fromInt(int i){
        N res= zero();
        while(i --> 0){ res= res.succ(); }
        return res;
    }
}

```

Just to print/test

Now we can look
at it




```

public interface N {
    static N zero() { return new N({}); }
    default N add(N other){ return other; }
    default N times(N other){ return this; }
    default N succ(){
        N pred= this;
        return new N(){
            public N add(N other){ return pred.add(other).succ(); }
            public N times(N other){ return pred.times(other).add(other); }
            public int toInt(){ return pred.toInt()+1; }
        }; }
    default int toInt(){ return 0; }
    static N fromInt(int i){
        N res= zero();
        while(i --> 0){ res= res.succ(); }
        return res;
    }
}

```



Zero.toInt == 0



```

public interface N {
    static N zero() { return new N({}); }
    default N add(N other){ return other; }
    default N times(N other){ return this; }
    default N succ(){
        N pred= this;
        return new N(){
            public N add(N other){ return pred.add(other).succ(); }
            public N times(N other){ return pred.times(other).add(other); }
            public int toInt(){ return pred.toInt()+1; }
        }; }
    default int toInt(){ return 0; }
    static N fromInt(int i){
        N res= zero();
        while(i --> 0){ res= res.succ(); }
        return res;
    }
}

```

Succ.toInt ==
recursive call + 1



```

public interface N {
    static N zero() { return new N({}); }
    default N add(N other){ return other; }
    default N times(N other){ return this; }
    default N succ(){
        N pred= this;
        return new N(){
            public N add(N other){ return pred.add(other).succ(); }
            public N times(N other){ return pred.times(other).add(other); }
            public int toInt(){ return pred.toInt()+1; }
        }; }
    default int toInt(){ return 0; }
    static N fromInt(int i){
        N res= zero();
        while(i --> 0){ res= res.succ(); }
        return res;
    }
}

```



In fromInt we can simply start from zero and then

use the 'runs to zero' operator to iterate up to 'i'.



You have seen
the 'runs to zero'
operator, right?





You have NOT
seen the 'runs to
zero' operator?

This is
unforgivable!

The runs to zero operator

```
if( stuff --> 0 ){...}
```

```
while( stuff --> 0 ){...}
```

- You DID learn about it in the first year, right?
- It is in C, C++, C#, Java, Python, Javascript, PHP, Swift and many more languages....

The runs to zero operator

```
if( stuff --> 0 ){...}
```

```
while( stuff --> 0 ){...}
```

- You DID learn about it in the first year, right?
- It is in C, C++, C#, Java, Python, Javascript, PHP, Swift and many more languages....
- Really, you have never seen it?

The runs to zero operator

```
if( stuff --> 0 ){...}
```

```
while( stuff --> 0 ){...}
```

- Troll lol lol lol lol....
Trolllololala!
Trolllololala!



The runs to zero operator

```
if( stuff-- > 0 ){...}
```

```
while( stuff-- > 0 ){...}
```

- Troll lol lol lol lol....
Trolllololala!
Trolllololala!



The runs to zero operator

```
if( stuff -- > 0 ){...}
```

```
while( stuff -- > 0 ){...}
```

- It is just two operators in sequence

<https://stackoverflow.com/questions/1642028/what-is-the-operator-in-c>

Don't forget about the Extra Long Arrow operator which moves twice as fast, x ----> 0

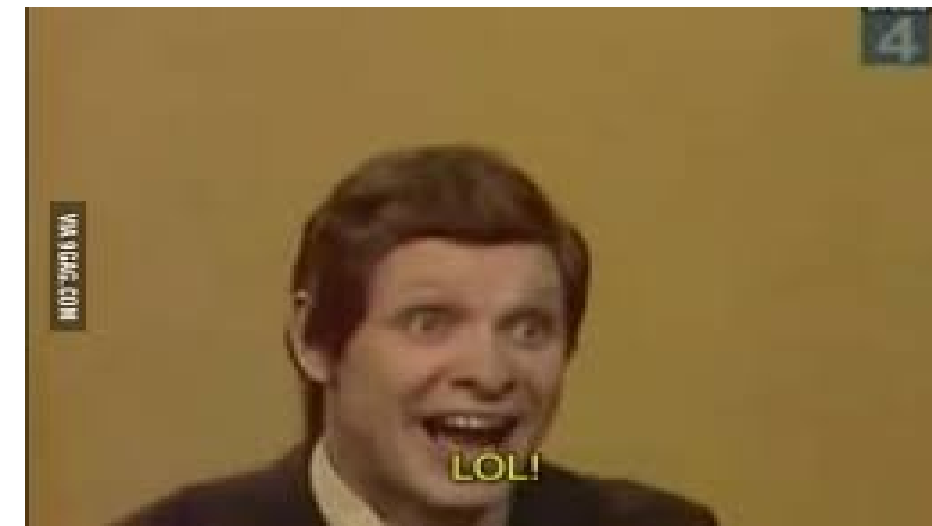
- If you have not seen this page, you need to read it now, it is gem!

The runs to zero operator

```
if( stuff --> 0 ){...}
```

```
while( stuff --> 0 ){...}
```

- However,....
 - It works.
 - It reads well.
 - It is fun.
 - It is maintainable.
 - It has localized impact on the code.
 - Would you use it, even if it is basically just a joke?
- If enough people chose to use it, would it still be just a joke?

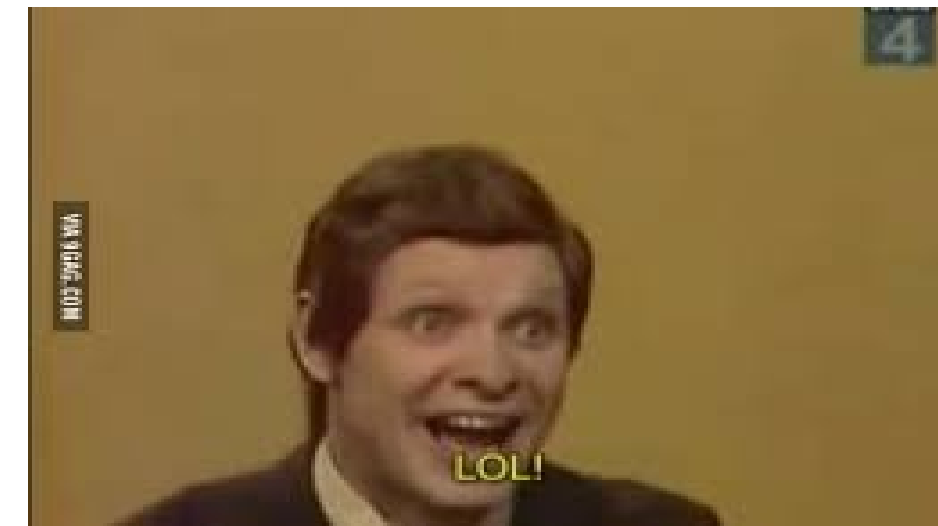


The runs to zero operator

```
if( stuff --> 0 ){...}
```

```
while( stuff --> 0 ){...}
```

- However,....
 - It works.
 - It reads well.
 - It is fun.
 - It is maintainable.
 - It has localized impact on the code.
 - Would you use it, even if it is basically just a joke?
- If enough people chose to use it, would it still be just a joke?
- Who is laughing now?
- Yes... many of the best programming techniques are born as 'jokes', 'tricks', 'challenges' or 'dares'



The biggest 'joke': dynamic dispatch

- Imperative mindset:
 - When you call a method in a certain point is because you know what this method is doing and you want that behavior to apply here.
- Dynamic dispatch: You have no idea what this method is going to do.
If you have no idea what this method does, why would you ever want to call it?
- The OO Programming Paradigm: programming where the most common and the most fundamental operation is the dynamically dispatched method call, so that you will have no clue what a method is actually going to do.
- It looks insane, right?

**However.... Everyone is using it.
May be it is not insane?**

Lab Week3&4 model solutions

```
class Range {
    static void check(double val, double min, double max){
        if (val < min || val > max){ throw new IllegalArgumentException(); }
    }
}

record GeoPoint(double latitude, double longitude){
    public String toString(){ return toString(8); }
    public String toString(int precision){
        if (precision < 0){ throw new IllegalArgumentException(); }
        String template =
            "[latitude: %."+precision+"f, longitude: %."+precision+"f]";
        return template.formatted(latitude, longitude);
    }
}

GeoPoint{
    Range.check(latitude, -90d, 90d); // Code reuse here!
    Range.check(longitude, -180d, Math.nextDown(180d));
}

GeoPoint average(GeoPoint gp){
    double avgLat= (this.latitude + gp.latitude) / 2d;
    double avgLon= (this.longitude + gp.longitude) / 2d;
    double longDiff= Math.abs(this.longitude - gp.longitude);
    if (longDiff > 180){ avgLon += (avgLon < 0) ? 180 : -180; }
    // If the points are on opposite sides of the anti-meridian
    return new GeoPoint(avgLat, avgLon);
}
}
```

Lab Week3&4 model solutions

```
record GeoBox(GeoPoint ne, GeoPoint sw){
  GeoPoint se(){ return new GeoPoint(sw.latitude(),ne.longitude()); }
  GeoPoint nw(){ return new GeoPoint(ne.latitude(),sw.longitude()); }
  GeoPoint center(){ return ne.average(sw); }
  GeoBox{
    Range.check(ne.latitude(),-80,80);
    Range.check(sw.latitude(),-80,80);
    var vSpan = ne.latitude() - sw.latitude();
    Range.check(vSpan,0,30);
    var hSpan = ne.longitude() - sw.longitude();
    if (hSpan >= 0){ Range.check(hSpan,0,30); }
    else { Range.check(hSpan,-360,-330); }
  }
}
```

Lab Week3&4 model solutions

```
record Street(GeoBox boundingBox, String name, int length) {
    Street{
        Objects.requireNonNull(boundingBox);
        Objects.requireNonNull(name);
        if (length < 0) { throw new IllegalArgumentException(); }
    }
    public String toString(){ return name; }
}

record Address(Street street, City city, String number){
    Address{
        Objects.requireNonNull(street);
        Objects.requireNonNull(city);
        Objects.requireNonNull(number);
        var invalidStreet= !city.streets().contains(street);
        if(invalidStreet){ throw new IllegalArgumentException(); }
    }
}
```


Lab Week3&4 model solutions

```
class City{
    private String name;
    private int population;
    private HashSet<Street> streets= new HashSet<>();
    public void population(int population){
        if (population < 0) { throw new IllegalArgumentException(); }
        this.population= population;
    }
    public String name(){ return name; }
    public int population(){ return this.population; }
    public void addStreet(Street s){ this.streets.add(s); }
    public Set<Street> streets(){ return Collections.unmodifiableSet(streets); }
    public City(String name, int population){
        this.name= Objects.requireNonNull(name);
        population(population);
    }
    public String toString() { return name; }
}
```

```
interface Building{
    GeoBox boundingBox();
    Address primaryAddress();

    default Set<Address> secondaryAddresses(){ return Set.of(); }

    static Building of(GeoBox boundingBox, Address primary, Set<Address> secondary){
        return new BigBuilding(boundingBox,primary,Set.copyOf(secondary));
    }

    static Building of(GeoBox boundingBox, Address primary){
        return new SingleAddressBuilding(boundingBox,primary);
    }

    static void checkOverlap(GeoBox box1, GeoBox box2){
        double s= Math.max(box1.sw().latitude(), box2.sw().latitude());
        double w= Math.max(box1.sw().longitude(), box2.sw().longitude());
        double n= Math.min(box1.ne().latitude(), box2.ne().latitude());
        double e= Math.min(box1.ne().longitude(), box2.ne().longitude());
        new GeoBox(new GeoPoint(n,e),new GeoPoint(s,w));//will fail if empty
    }
}
```

```
record BigBuilding(
    GeoBox boundingBox, Address primaryAddress, Set<Address> secondaryAddresses)
    implements Building{
BigBuilding{
    Building.checkOverlap(boundingBox, primaryAddress.street().boundingBox());
    if (secondaryAddresses.isEmpty()){ throw new IllegalArgumentException(); }
    for (var s:secondaryAddresses){
        Building.checkOverlap(boundingBox, s.street().boundingBox());
    }
}
}
```

```
record SingleAddressBuilding(GeoBox boundingBox, Address primaryAddress) implements Building{
    SingleAddressBuilding{
        Building.checkOverlap(boundingBox, primaryAddress.street().boundingBox());
    }
}
```